PROCESSES IN OPERATING SYSTEMS

1. Introduction to Processes

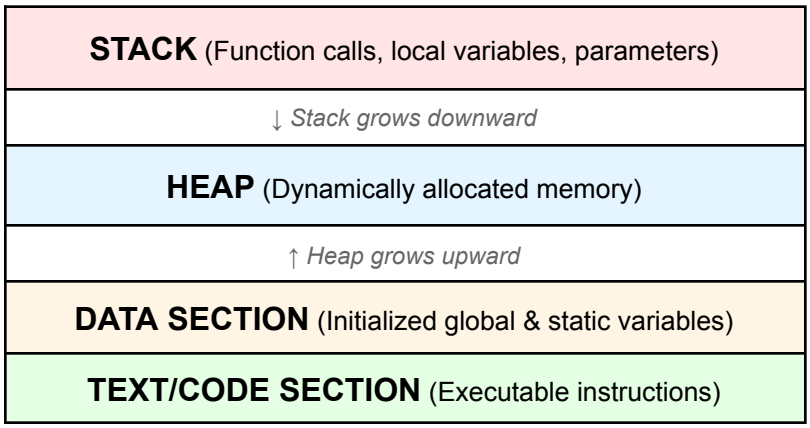
A process is a program in execution. It represents the basic unit of work in a modern computing system. Understanding how a program transforms into a process and how it executes is fundamental to operating systems.

1.1 What is a Process?

A process consists of:

- Program Code (Text Section)
- Current Activity (Program Counter, Processor Registers)
- Stack (Temporary Data - function parameters, return addresses, local variables)
- Data Section (Global Variables)
- Heap (Dynamically allocated memory during runtime)

Diagram 1: Process Memory Structure



1.2 Program vs Process

Aspect	Program	Process
Nature	Passive entity	Active entity
Location	Stored on disk	Loaded in memory
Lifespan	Permanent (until deleted)	Temporary (while executing)
Resource Usage	Disk space only	CPU, memory, I/O devices

Aspect	Program	Process
Multiplicity	One file	Multiple instances possible

2. How a Program Resides on Hard Disk

When a program is stored on a hard disk, it exists as an executable file. This file contains all the necessary information to create a process when executed.

2.1 Executable File Format

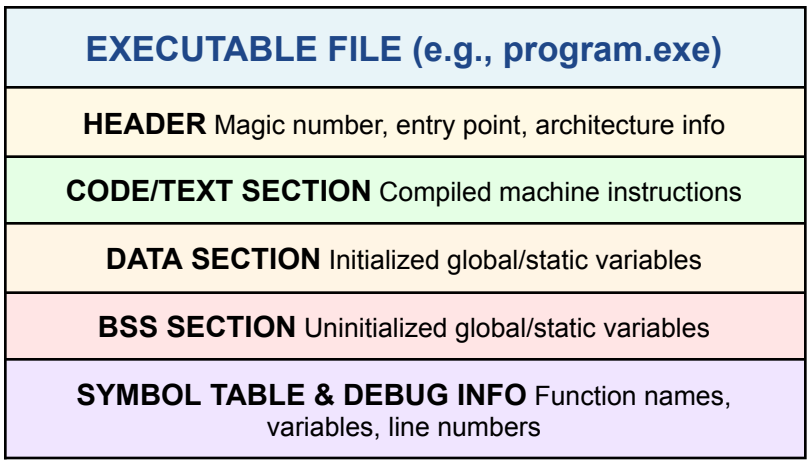
Common executable formats include:

- EXE (Windows Executable)
- ELF (Executable and Linkable Format - Linux/Unix)
- Mach-O (macOS)

2.2 Components of Executable File

- **Header:** Contains metadata about the program (entry point, architecture, etc.)
- **Code Section:** Contains the compiled machine code instructions
- **Data Section:** Contains initialized global and static variables
- **BSS Section:** Contains uninitialized global and static variables
- **Symbol Table:** Information for debugging and linking

Diagram 2: Program Storage on Hard Disk



```c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process
        printf("Child process: PID = %d\n", getpid());
    } else {
        // Parent process
        printf("Parent process: PID = %d, Child PID = %d\n", getpid(),
pid);
    }

    return 0;
}
```

Memory Section	**Code Elements Mapped**	**Runtime Behavior**
Stack	Local variable pid	Created when main() is called, contains local variables and function call information
Heap	None (no dynamic allocation)	Would contain malloc() allocations if present
.bss	None	Empty section for uninitialized globals
.data	None	Empty section for initialized globals
.rodata	String literals: "fork failed" "Child process: PID = %d\n" "Parent process: PID = %d, Child PID = %d\n"	Read-only, shared between parent and child processes
.text	Entire main() function: 1. fork() call 2. if-else logic 3. printf() calls 4. getpid() calls 5. return statements	Read-only executable code, shared between parent and child
ELF Header	Program metadata	Loaded by OS to set up process (Executable and Linkable Format) (ELF) header

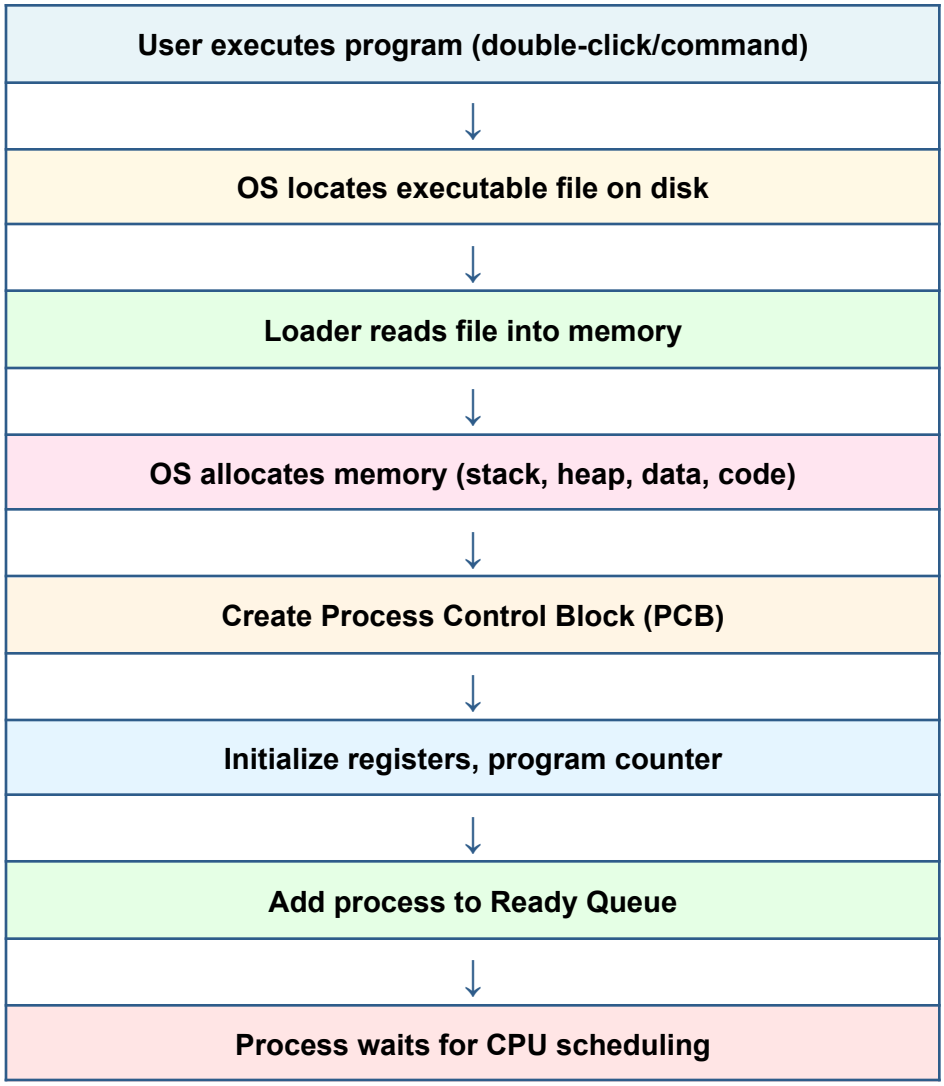
3. From Program to Process: The Transformation

When you execute a program, the operating system performs several steps to transform the passive program file into an active process.

3.1 Process Creation Steps

1. **User Initiates Execution:** Double-click, command line, or system call
2. **Load Executable into Memory:** OS reads executable file from disk and loads it into RAM
3. **Memory Allocation:** OS allocates memory for code, data, stack, and heap
4. **PCB Creation:** Process Control Block is created with process metadata
5. **Initialize Process State:** Set program counter to entry point, initialize registers
6. **Add to Ready Queue:** Process is placed in ready state, waiting for CPU

Diagram 3: Process Creation Flow



4. Process Control Block (PCB)

The Process Control Block is a data structure maintained by the operating system for every process. It contains all the information needed to manage the process.

4.1 PCB Components

PCB Field	Description	Example
Process ID (PID)	Unique identifier for the process	Parent: 1000 Child: 1001
Process State	Current state (New, Ready, Running, Waiting, Terminated)	**Parent:** Running → Ready (when waiting for child) **Child:** New → Ready → Running → Terminated
Program Counter	Address of next instruction to execute	0x400510 (address of printf("Child...")) Example: Child's PC points to first instruction after fork()
CPU Registers	Contents of all processor registers	RAX: 1001 (fork return value in parent) RAX: 0 (fork return value in child) RBP: 0x7fffffffee240 (stack frame pointer) RSP: 0x7fffffffee230 (stack pointer)
Memory Management	Page tables, segment tables, memory limits	Page Table Base: 0x8000 Text Segment: 0x400000-0x400fff (read-only) Data Segment: 0x600000-0x600fff Stack Limit: 0x7fffffff0000
Scheduling Info	Priority, scheduling queue pointers, time quantum	Priority: 120 (normal priority) Time Quantum: 100ms Queue Pointer: Ready queue position 5
I/O Status	List of open files, I/O devices allocated	0: stdin (terminal) 1: stdout (terminal) 2: stderr (terminal) Open Files: /dev/tty1
Accounting Info	CPU time used, time limits, process creation time	Creation Time: 14:30:25.123456 CPU Time Used: 12.5ms

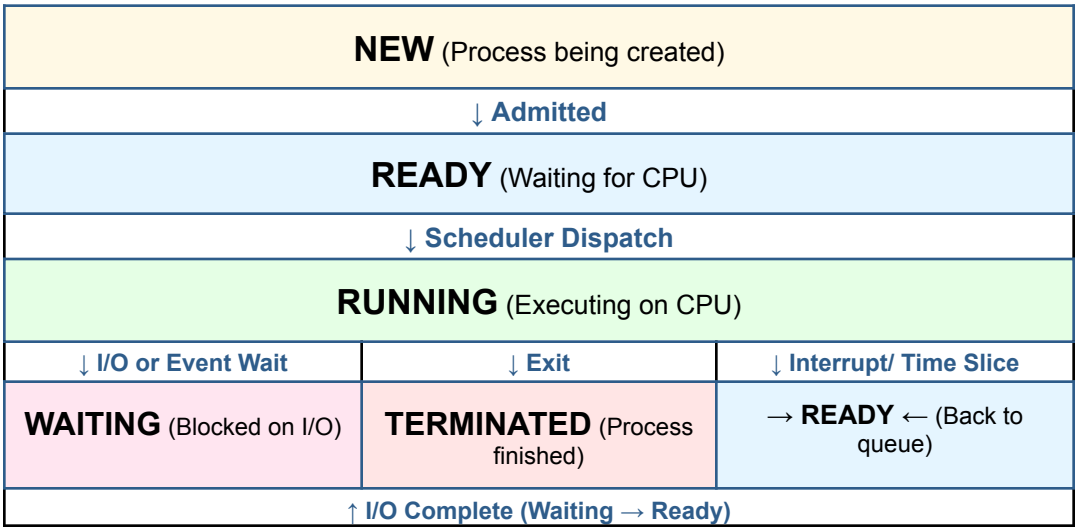
5. Process States and Life Cycle

Throughout its lifetime, a process moves through different states from creation to termination.

5.1 Five-State Process Model

7. **New:** Process is being created
8. **Ready:** Process is waiting to be assigned to a processor
9. **Running:** Instructions are being executed
10. **Waiting/Blocked:** Process is waiting for some event (I/O completion, signal, etc.)
11. **Terminated:** Process has finished execution

Diagram 4: Process State Transition Diagram



5.2 State Transitions Explained

Transition	Explanation
New → Ready	Process creation complete, added to ready queue
Ready → Running	Scheduler selects this process for execution
Running → Ready	Time quantum expired or higher priority process arrives
Running → Waiting	Process needs I/O or waits for an event
Waiting → Ready	I/O completed or event occurred
Running → Terminated	Process completes execution or is killed

6. Process Execution

Once a process is in the running state, the CPU executes its instructions. The process may be interrupted, scheduled out, or complete its execution.

6.1 CPU Scheduling

The CPU scheduler determines which ready process should be executed next. Common scheduling algorithms include:

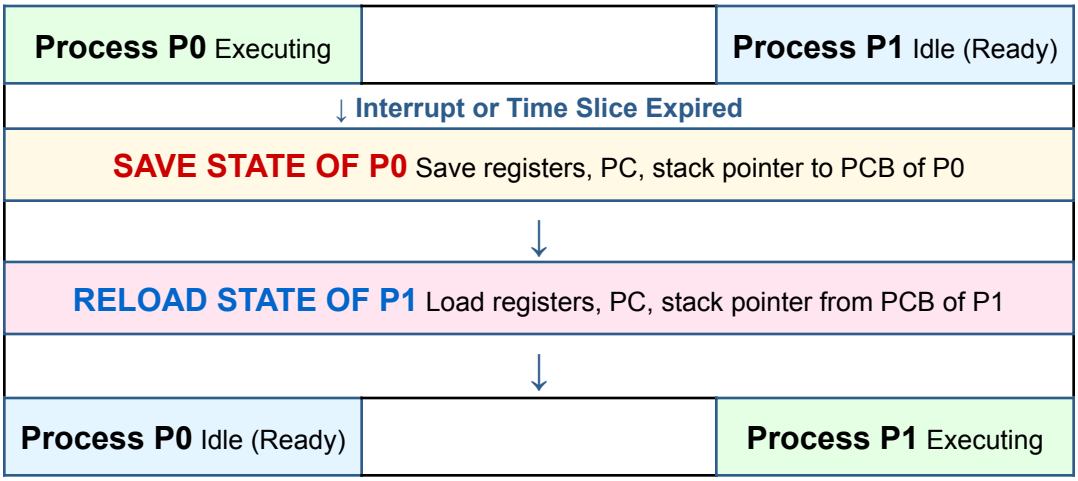
- First-Come, First-Served (FCFS)
- Shortest Job First (SJF)
- Priority Scheduling
- Round Robin (RR)
- Multilevel Queue Scheduling

6.2 Context Switching

When the CPU switches from executing one process to another, it performs a context switch. This involves:

12. Saving the state (context) of the current process in its PCB
13. Loading the state of the next process from its PCB
14. Resuming execution of the new process

Diagram 5: Context Switching



7. Process Termination

A process terminates when it completes execution or is explicitly killed. Upon termination, the OS performs cleanup operations.

7.1 Reasons for Termination

- **Normal Exit:** Process completed successfully
- **Error Exit:** Process encountered an error and terminated

- **Fatal Error:** Process performed illegal operation (division by zero, segmentation fault)
- **Killed by Another Process:** Process terminated by parent or operating system

7.2 Termination Process

15. Process executes exit system call (voluntary) or receives termination signal
16. OS deallocates all resources held by the process
17. Memory pages are freed and returned to the system
18. Open files are closed
19. PCB is removed from process table
20. Parent process is notified (if waiting)

Diagram 6: Complete Process Life Cycle